ARAŞTIRMA SORULARI

1.Syntax error,runtime error ve logic error arasındaki farklar nelerdir?

-Syntax error(sözdizimi hatası):
–Kod yazarken o dilin dil bilgisi(gramer)kurallarına uymadığınızda gerçekleşir.Bilgisayar kodu anlamlandıramadığı için programı çalıştırmayı reddeder.

-Neden olur?:
–Yanlış anahtar kelime kullanımı,unutulan noktalı virgüller veya parantezler.
örnek(Python):
python
print(“Merhaba” # Parantez kapatılmadı

-Runtime Error(çalışma zaman hatası)
–Kodun yazımında hiçbir hata yoktur,bu yüzden program çalışmaya başlar.Ancak çalışma esnasında bilgisayarın yerine getiremeyeceği veya izin verilmeyen bir komutla karşılaşınca program aniden çöker(crash olur).

-Neden olur?:
–Geçersiz matematiksel işlemler,yetkisiz hafıza/dosya erişimleri veya bellek tüketimi.
Örnek(Python):
python
sayi = 10 / 0 # Bir sayı sıfıra bölünemez (ZeroDivisionError)

-Logic Error(mantık hatası/bug):
–Program dil bilgisi açısından kusursuzdur ve hiçbir aşamada çökmeden başarıyla tamamlanır.Ancak program,sizin istediğiniz şeyi değil,ona yazdığınız şeyi yapar.Yani sonuçlar tamamen yanlıştır.

-Neden olur?:
–Yanlış algoritma kurmak,Yanlış operatör(örn < yerine >) kullanmak veya yanlış formüller uygulamak.
Örnek(python):
python
#Amaç iki sayının ortalamasını bulmak

ortalama = 10 + 20 / 2 # İşlem önceliği yüzünden sonuç 30 yerine 20 çıkar! (Doğrusu: (10 + 20) / 2)

2.Hata mesajı okurken hangi bilgilere bakılır?

Bir hata mesajını okurken ve çözerken doğru adımları atmak için "hata türü,hata kodu,mesaj metni ve hatanın gerçekleştirdiği yer"gibi kritik bilgilere bakılır.Hata mesajları,sistemin nerede ve neden durduğunu gösteren birer haritadır.

Bir hata mesajını incelerken bakmanız gereken temel unsurlar şunlardır:

1.Hata Türü veya Kategorisi(Error Type/Category):
–Hatanın hangi sistem,dil veya kütüphane tarafından fırlatıldığını belirler.Sorunun genel çerçevesini çizmenizi sağlar.
Örnekler:“TypeError,NullReferenceException,404 Not Found,SyntaxError”.

2.Hata kodu(Error Code):
–Yazılımların veya işletim sistemlerinin belirli hataları tanımlamak için kullandığı benzersiz(unique)kodlardır.İnternette arama yaparken en kesin sonuçları bu kodlar verir.
Örnekler:“0x80070005(Windows erişim reddedildi hatası),403 (Yetkisiz erişim),ERR_CONNECTION_TIMED_OUT”.

3.Mesaj metni/açıklama(error message/description):
–Hatanın neden kaynaklandığını insan diline en yakın şekilde özetleyen kısımdır.Sistem size neyin eksik veya yanlış olduğunu doğrudan söyler.
Örnkler:“disk full(disk dolu),user not Found(kullanıcı bulunamadı),string expected,got integer(metin bekleniyordu,sayı geldi)”.

4.Konum bilgisi(dosya adı ve satır numarası):
–Özellikle yazılım geşiştirirken hatanın tam olarak projenin hangi dosyasında ve kaçıncı satırında meydana geldiğini gösteririr.
Örnekler:“at index.js:42:15(index.js dosyanın 42.satırının 15.karakterinde)”.

5.Yığın izi(strack trace):
–Hatanın gerçekleştiği ana kadar programın hangi fonsksiyonları hangi sırayla çağırdığını gösteren geçmiş dökümüdür.Hatayı tetikleyen asıl ilk kök nedeni(ilk tetikleyici)bulmak için aşağıdan yukarıya veya yukrıdan aşağıya doğru okunur.

3.Debugger ne işe yarar?

Yazılımcıların kaynak kodarında bulunan hataları(bug)tespit etmek,izole etmek ve çözmek için kullandığı temel bir yazılım aracıdır.

TEMEL İŞLEVLERİ

-Adım adım çalıştırma(step-by-step execution):
–Kodun tamamını bir kerede çalıştırmak yerine,kod satırlarını tek tek(satır satır)ilerleterek programın akışını kontrol etmeyi sağlar.

-Kırılma noktaları(Breakpoint):
–Kodun belirli bir satırında çalışmayı durdurarak o ana kadar neler olduğunu incelemeye olanak tanır.

-Değişken takibi(veriable ınspection):
–Program çalışırken değişkenlerin anlık değerlerini,hafızada nasıl değiştiğini ve aldıkları parametreleri canlı olarak gösteririr.

-Çağrı yığını(call stack)incelemesi:
–Hatanın hangi fonksiyonun içinden tetiklendiğini ve hangi yollardan geçerek buraya ulaşıldığını gösterir.

SAĞLADIĞI AVANTAJLAR

-Hızlı çözüm:
–Hangi satırda ve neden hata oluşturduğunu tahmin etmek yerine net bir şekilde görmeyi sağlar.

-Zaman tasarrufu:
–“print” veya "log"yazarak kodu sürekli yeniden başlatma ihtiyacını azaltır.

-Kod analizi:
–Sadece hata bulamk için değil,karmaşık bir kodun mantığını ve çalışma yapısını daha iyi anlamak için de kullanılır.

4.Breakpoint(kırılma noktası) nedir?

Kodun belirli bir satırına koyulan işaretçidir.Program çalıştırıldığında kod o satıra geldiğinde otomatik olarak durur.

Amacı:
–Geliştiricinin program o andeyken değişkenlerin değerlerini incelemesine,hafızayı kontrol etmesine ve kodu adım adım (step by step)çalıştırarak hata ayıklamasına(debugging)olanak tanır.

5.Step over/step into gibi kavramlar ne anlatır?

-Step Over(adım atla/üzerinden geç)
-Ne yapar?:
–Kodun bulunduğunuz satırını çalıştırır ve doğrudan bir sonraki satıra geçer.

-Fonksiyon davranışı:
–Eğer o satırda başka bir fonksiyon çağrısı varsa(örneğin hesapla()), o fonksiyonun içine girmez.Fonksiyonu arka planda tamamen çalıştırır ve sonucu alıp bir alt satırda bekler.

-Ne zaman kullanılır?:
–Çağrılan fonksiyonun doğru çalıştığından eminseniz ve onun detaylarıyla vakit kaybetmeden mevcut kod bloğundan devam etmek istiyorsanız kullanılır.

-Step Into(içine gir):

-Ne yapar?:
–Kodun o satırındaki detayların en derinine iner.

-Fonksiyon davranışı:
–Eğer o satırda bir fonksiyon çağrısı varsa,program o fonksiyonun ilk satırına dallanır ve orada durur.

-Ne zaman kulanılır?:
–Hatayı bir fonksiyonun kendi içinde arıyorsanız ve o fonksiyon çağrıldığında içeride tam olarak ne olduğunu adım adım görmek istiyorsanız kullanılır.

6.Test neden yapılır?

Geliştirilen bir ürünün doğru, güvenli ve sorunsuz çalıştığından emin olmak için yapılır.

7.Edge case medir?

Bir yazılımın,sistemin veya algoritmanın normal,beklenen ve tipik çalışma aralığının en uç noktalarında ortaya çıkan sıra dışı durumdur.

TEMEL ÖZELLİKLERİ NELERDİR?:

-Uç Değerler:
–Bir parametrenin alabileceği minimum veya maksimum sınırlarda(örneğin sıfır,negatif sayılar veya sistemin kaldırabileceği en yüksek kapasite)meydana gelir.

-Beklenmedik davranışlar:
–Günlük kullanımda çok nadir görülse de,test edilmediğinde sistemin çökmesine,kilitlenmesine veya hatalı sonuç üretmesine yol açabilir.

-Sınırlarda çalışma:
–Sistemin operasyonel sınırlarında gerçekleşir.

GÜNLÜK HAYATTAN VE YAZILIMDAN ÖRNEKLER:

-E-ticaret sepeti:
–Bir kullanıcının sepete 1 ürün eklemesi veya hiç ürün eklemeden(sepet boşken)ödeme yapmaya çalışması tipik bir durumdur;ancak aynı anda 10.000 farklı kitap satın almaya çalışması bir edge case'dir.

-Kiralama sistemi:
–Bir araç kiralama sisteminde günlük ücret hesaplanırken,müşterinin aracı 0 gün kiralaması veya negatif gün sayısı girmesi uç durumlara örnektir.

-Kullanıcı kaydı:
–Bir üyelik formunda e-posta adresinin çok kısa olması normal bir senaryodur;ancak e-postanın bin karakter uzunluğunda olması ya da Sadece özel karakterlerden oluşması bir edge case oluşturur.

Gün sonunda teslim edeceğiniz not:

1.Üç hata türü için ayrı ayrı örnek senaryo bulun.

-Syntax Error(yazım hatası):
–Bir öğrenci,C# programında ekrana "Merhaba Dünya"yazdırmak ister.Ancak kodu yazarken noktalı virgülü(;)unutursa program derlenirken hata verebilir.

-Runtime Error(çalışma zamanı hatası):
–Bir öğrenci,iki sayıyı birbirine bölmek isteyen bir program yazar.Ancak notları topladıktan sonra toplamı öğrenci sayısı yerine yanlışıkla 2'ye bölerse program çalışır ama sonuç yanlış çıkar.

-Logic Error(mantık hatası)
–Bir öğrenci,sınav notlarının ortalamasını hesaplayan bir program yazar.Ancak notları topladıktan sonra toplamı öğrenci sayısı yerine yanlışıkla 2'ye bölerse program çalışır ama sonuç yanlış çıkar.

-Kısaca farkları:
–Syntax Error:
-Kodun yazım kurallarına uyulmaması.
–Runtime Error:
-Program çalışırken ortaya çıkan hata.

–Logic Error:
-Program çalışmasına rağmen yanlış sonuç üretmesi.

2.Bir hata geldiğinde uygulayacağınız adımları maddeler halinde yazın:

-Hatayı kaydedin ve tanımlayın:
–Hata kodunu,tam hata mesajını ve hatanın gerçekleştiği ekranı not edin.

-Hata ortamını belirleyin:
–Hatanın hangi işletim sistemi,tarayıcı,cihaz veya uyguulama sürümünde oluştuğunu tespit edin.

-Adımları yeniden üretin(Reproduce):
–Hataya sebep olan kullanıcı adımlarını sırasıyla uygulayarak hatayı istikrarlı bir şekilde tekrar ortaya çıkarmaya çalışın.

-Logları ve günlükleri inceleyin:
–Uygulama,sunucu veya veri tabanı loglarını(günlük kayıtlarını)inceleyerek arka planda dönen detayları kontrol edin.

-Kök nedeni analiz edin(Root cause):
–Hataya neyin sebep olduğunu(kod hatası,eksik bağımlılık,veri tabanı çökmesi,yetkilendirme sorunu vb.)bulun.

-Çözüm geliştirin ve test edin:
–Hatayı düzeltecek kodu yazın veya yapılandırma değişikliğini yapın.Çözümü önce güvenli(staging)ortamında deneyin.

-Değişikliği canlıya alın(deploy):
–Testlerden başarıyla geçen çözümü canlı sisteme aktarın.

-Doğrulayın ve izleyin:
–Hatayı canlı ortamda tekrar kontrol edin ve sisteme izleme(monitoring)araçlarıyla benzer bir hatanın tekrarlamadığından emin olun.

-Dokümantasyon yapın:
–Hatayı,nedenini ve çözümünü gelecekte benzer sorunları hızla çözebilmek adına bilgi tabanına(knowledge base)kaydedin.

3.“Çalışıyor” demenin neden tek başına yeterli olmadığını araştırıp açıklayın: